

Challenges of the CDIF Production Work

Joachim Wackerow - 2026-09-08

There are several challenges in the CDIF production process that have made the work more difficult and require solutions. This document describes the experiences while this work and mentions suggestions for possible solutions.

Overall, improving documentation for various purposes and clear rules in handling the CDIF resources are the key points.

Suggestions on individual topics may overlap with suggestions on other topics.

List of Challenges

1 UML Standard and Tool Incompatibility.....	1
2 Differences Between RDF and UML Concepts.....	2
3 Validation with JSON Schema and SHACL.....	2
4 Unclear Overall Context and Source of Truth.....	4
5 CDIF.....	4
6 Acceleo and the Model-to-Text Tooling.....	5

1 UML Standard and Tool Incompatibility

a Challenges

The goal was developing a UML model confirming UCMIS (UML Class Model Interoperable Subset) and using the tools Enterprise Architect and Eclipse Modeling (QVTo and Acceleo). A roundtrip between EA and Eclipse should be possible.

UCMIS was scoped for DDI-CDI; CDIF's needs pushed past that frame, forcing UCMISM2T-specific, non-standard extensions — meaning the tool itself is now somewhat bespoke rather than a clean implementation of a portable standard. Nevertheless, the tool can still process other models that are UCMIS-compliant—such as DDI-CDI.

The CDIF model is a UML model which is UCMIS compliant and available as Canonical XMI. It can be imported into common UML tools. It uses some feature beyond UCMIS and UML which can recognized only by custom software. Typical UML approaches such as “tagged values” and stereotypes are avoided, as they generally work reliably only within a specific UML tool.

- PackageInformation (metadata-as-DataType) – used for metadata per package
- Keywords embedded in comment bodies – used for selected attributes per profile
- The association-naming convention - composite association name (Subject_predicate_Object), also used for DDI-CDI
- class attributes with different namespace – like cdi:isStructuredBy. Names with colons are allowed but UML doesn't have the concept of a different namespace in an attribute.
- Information for domainIncludes / rangeIncludes of schema.org – *work not finished yet*

b Suggestions

Documentation on of exactly where the CDIF model and the tool UCMISM2T depart from standard UCMIS.

2 Differences Between RDF and UML Concepts

a Challenges

RDF and UML don't map onto each other cleanly, and several of the resulting gaps have no clean resolution - only a choice of trade-off:

- RDF assumes an open world (absence of a statement doesn't mean it's false); UML/object-oriented modeling generally assumes a closed one.
- RDF properties are defined globally, independent of any single class; UML has no equivalent concept, since properties belong to the class that owns them.
- Schema.org's `rangeIncludes/domainIncludes` pattern — where a property can apply across many different classes rather than belonging to one — doesn't map onto UML's typing model at all.

b Suggestions

These gaps generally can't be "fixed," only made into visible, deliberate decisions rather than points of recurring confusion. A short, written note per gap — what the trade-off is and which choice was made, and why — turns each one into a known, referenceable decision instead of something re-litigated by whoever runs into it next.

Describe clearly which CDIF components are intended for the use in an open-world assumption (OWA) context and which in an closed-world assumption (CWA) context.

3 Validation with JSON Schema and SHACL

a Challenges

What is the goal of the validation? Which kind of design policy should be taken?

The general, theoretical difference

RDF/OWL's native semantics are open-world (OWA) — absence of a statement means "unknown," not "false." A graph can always be extended with more triples from anywhere, and nothing assumes the graph is complete. But *validation* is inherently a closed-world-flavored activity — "this must have at least one X," "only these properties are allowed" — these assert something is wrong when a condition isn't met, which only makes sense if you're willing to treat what's *absent* as meaningful.

- **JSON Schema** was designed for validating plain JSON documents — no native concept of graphs, classes, or `rdfs:subClassOf`. Its spec default also allows extra properties unless `"additionalProperties": false` is set, but in practice, most real-world JSON Schema usage (API contract validation especially) sets it explicitly, since the usual question being asked ("does this payload match my contract") is inherently closed-world in spirit.

- **SHACL** is designed to let you apply CWA-style validation to a specific, given RDF graph, without changing RDF's own OWA semantics globally — it validates a bounded "data graph" against a "shapes graph" at a point in time, and by the spec's own default, extra properties are *allowed* unless a shape explicitly says `sh:closed true`.

Current approach in SHACL and JSON Schema generated by UCMISM2T:

-
- **JSON Schema:** `additionalProperties: false` appears in the class and datatype shape templates.
- **SHACL:** `sh:closed true` appears in the equivalent class and datatype shape templates too.

So **both** generated artifacts are configured to reject any property not explicitly declared in the shape — not just JSON Schema by convention, but SHACL too, via an explicit, deliberate opt-in that goes against SHACL's own more permissive default. This is a genuinely consistent choice across both formats, not an accidental asymmetry.

This connects directly to the questions — mixing vocabularies from different sources within profiles, and whether "existing software is prepared for this." With both validators set to closed/strict:

- A payload with one extra property from an unlisted vocabulary — even a perfectly legitimate RDF extension, fully valid under the data model's own open-world semantics — would **fail validation against both** the generated JSON Schema and the generated SHACL shapes. Neither format is currently acting as the more permissive, OWA-respecting option; the project has consistently chosen strictness in both.
- This is a defensible, common choice for **conformance testing** — "does this payload match exactly what this profile declares" is often exactly the question worth asking, especially for a profile meant to define a specific, closed contract for implementers. But it does mean this project's validation posture, as currently generated, doesn't lean on SHACL's typical advantage over JSON Schema (better compatibility with open, extensible, multi-vocabulary graphs) — that advantage exists in SHACL's design, but isn't being used here.

b Suggestions

If the goal is "strict conformance to a named profile" (validating an implementer's payload against CDIFCore, say), the current closed-shape choice makes sense in both formats. If the goal is closer to "is this a valid, extensible RDF graph that at least satisfies these particular constraints" — the more OWA-native use case SHACL is actually built for — the current `sh:closed true` setting is working against that, and dropping it (while keeping it in JSON Schema, where document-shape validation is the more natural fit) might better reflect the actual difference in what each format is suited for. That's a modeling/governance decision, not a tool bug — but worth being an explicit choice rather than something that happened to end up symmetric by default.

4 Unclear Overall Context and Source of Truth

There's no settled answer to a basic question: is the UML model, the implementation guide, or the JSON Schema the authoritative definition of CDIF? This is a meaningfully different situation from DDI-CDI, where the model itself was built from the start to be the single source of truth. CDIF's current arrangement — several documents, none clearly subordinate to the others — is a root cause behind much of the friction while this work.

Here's a somewhat heretical question: What is the actual purpose of a UML model that uses parts of RDF vocabularies and whose generated artifacts are intended for RDF software, assuming an open world?

a Suggestions

The CDIF recommendations would benefit from further discussion of their technical architecture, including both the resources that comprise the recommendations and their practical application. Two complementary documents could emerge from this work: one describing the technical architecture of CDIF and another illustrating exemplary applications of CDIF.

Make a clear design decision regarding the question of the single source of truth, and document the resulting document structure. Provide a document that describes this concept, as well as the purpose of each resource within this structure and its dependencies on other resources. All changes should be made in a disciplined manner and in accordance with this structure.

This is related to topic 5.

5 CDIF

a Challenges

Extension from 1.0 to 1.1

CDIF was extended from version 1.0 to 1.1 while the work on the CDIF production. This was a moving target while developing the model and the tools. There is an impression that the documentation base could benefit from greater consistency and from further alignment with the conventions and expectations of a standards recommendation. This affects two dimensions, the divergence across profiles and across document types.

Disagreeing document types

Over time, various document types were cited as important sources of information: spreadsheets, JSON schema files, a Word document, and Markdown implementation guides. For a long time, it was unclear which document type was considered the authoritative source. The document types contained discrepancies in content.

The issue of discrepancies between the document types was not resolved until relatively late in the process, when it was determined that the Markdown-formatted implementation guides would serve as the authoritative source. The downside is that these are text documents that are not readily machine-readable.

Divergence to the claim of inventing nothing new

CDIF's stated goal of not inventing new constructs isn't fully realized in practice — some elements are effectively new inventions dressed in borrowed vocabulary.

Many classes are extensions of DDI-CDI (a related, earlier standard), which creates a real risk: changes to CDIF's extended classes could break existing DDI-CDI software that wasn't expecting them.

Separately, CDIF's "profiles" mix classes and properties from different external vocabularies (Schema.org, SKOS, DCAT, and others) in new combinations — fully legitimate under RDF's own philosophy, but untested against how forgiving or fragile real-world RDF-consuming software actually is when it encounters unfamiliar combinations. True RDF-compatible software should ignore unknown elements without crashing. However, this cannot always be expected in the context of data portals.

b Suggestions

Treat the "single source of truth" question (expanded in separate section) as the highest-leverage fix available for this area — much of the cross-document disagreement stems from not having settled this. This is related to topic 4.

Build a short, explicit map of which document derives from which, and which are hand-maintained versus generated.

Implementation guides should have a section which focuses on these extensions and unfamiliar combinations. This will be an important step for the appropriate implementation of CDIF profiles.

Using JSON-LD Framing allows to reshape the CDIF graph into the exact tree structure an existing application might expect. An implementation guide could describe this approach.

6 Acceleo and the Model-to-Text Tooling

a Challenges

The tool UCMISM2T was initially developed for DDI-CDI on the basis of Acceleo 3.7, later it was generalized in order for processing any model conforming to UCMIS (UML Class Model Interoperable Subset).

The current version of UCMISM2T is migrated to and built on Acceleo 4.2 in the hope achieving a robust model-to-text tool. Though, Acceleo is a still-maturing version of the Eclipse model-to-text transformation engine. Acceleo 3.7, the more established predecessor, has its own well-known problems and is not supported anymore — so this wasn't an unreasonable choice, but it does mean the project is absorbing the growing pains of a newer platform.

UCMISM2T is an Eclipse Acceleo 4.2 (AQL) model-to-text transformation tool that reads a single UML model and generates seven distinct machine-readable and human-readable artifacts from it: JSON-LD contexts, JSON Schema, SHACL shapes, ShEx schemas, XSD schemas, OWL/Turtle ontology, and Sphinx/RST documentation with embedded PlantUML diagrams. It supports both whole-model ("universe") generation and dependency-driven "profile" generation, where a named

subset of the model — a curated selection of classes reachable from a small set of explicit dependencies — is generated as a self-contained artifact set in each output format.

The tool's central architectural bet is that a single, well-annotated UML model can serve as the sole source of truth for a family of otherwise-divergent serialization formats, each with different structural capabilities (RDF's open-world graph model, JSON Schema's tree-shaped validation, XSD's simple/complex-content split, RST's document-oriented cross-referencing). Much of the tool's internal complexity exists specifically to bridge the gaps between what UML can express and what each target format can natively represent — polymorphic type unions, language-tagged literals, flexible object-or-reference attributes, and profile-scoped cross-referencing all required dedicated handling because no single UML construct maps cleanly onto all seven targets at once.

Three specific issues with Acceleo 4.2 have real, ongoing cost:

- The IDE's build process is intermittently broken, forcing a slower workaround: editing in the IDE but building from the command line.
- The IDE sometimes reports cascading false errors triggered by a single real error elsewhere in the code, making root causes hard to isolate. The only reliable mitigation found so far is making small, incremental changes and testing each one before moving on. This has been filed as an issue with the Acceleo project.
- Whitespace and indentation handling in Acceleo is stricter than the underlying OMG MOFM2T standard requires, and conflicts directly with output formats like RST (reStructuredText), where whitespace and indentation carry real structural meaning — similar to Python. This has caused concrete, hard-to diagnose build failures. Also filed upstream.

b Suggestions

- Write down the known Acceleo workarounds once, briefly, as a shared reference document: the small increment editing discipline, the exact indentation rules that matter, and the specific whitespace pitfalls already discovered. These are currently known only informally; documenting them once means they don't need to be painfully rediscovered by whoever encounters them next.
- Continue tracking the two filed upstream issues, and consider periodically checking whether newer Acceleo releases have addressed either — both represent real, recurring cost as long as they remain open.